CSE 211 — Theory of Computation

Nondeterminism and Nondeterministic Finite Automata

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Hakim Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — NFA design and NFA→DFA conversion in almost every paper
Lesson 6 — Nondeterminism and the NFA — guessing, computation trees, formal definition, DFA vs NFA
Lesson 7 — Designing NFAs — guess where a pattern starts, a split point, which condition holds
Lesson 8 — ε-Transitions and Removing Them — ε-NFA, ε-closure, δ′ and F′
Lesson 9 — NFA → DFA: the Subset Construction — lazy evaluation, ε-NFA → DFA, the proof, the 2^n blow-up
Quick Notes — NFA — revision bullets + key diagrams

NOTE Slide numbers refer to the PDF pages of Lec04 - NFA.pdf (109 pages; beamer builds, so ranges like sl. 8–10 are one slide). Every NFA in the solutions was run on all strings up to length 8 (5 for 3-letter alphabets) against the language definition, and every subset construction was generated by a program. 'Lazy evaluation' (older papers) = build only the subsets reachable from the start. Practice provenance: Actual = taken from a TF paper; Adapted = TF question with changed details; Constructed = new question in TF style.

Exam Map — What the TF Papers Ask

Session	NFA questions
2023-24	Q3(a) 3-state NFA for {a^n | n ≥ 1} ∪ {b^m a^k | m, k ≥ 0}; can fewer states work? (15) · Q4(a) NFA for {a}∗ such that removing one edge leaves an NFA for {a} (10)
2022-23	Q4(b) NFA: binary strings that begin and end with different symbols and have odd length (15)
2021-22	Q2(a) Why is DFA/NFA equivalence surprising and useful? (4) · Q3(b) NFA: binary strings with at least two 0s or exactly two 1s (10) · Q3(c) Convert the ε-NFA (q1, q2, q3 table) to a DFA (9)
2020 (Jan term)	Q1(c) ε-NFA (p, q, r table): general forms of accepted strings of length ≤ 3 (12) · Q2(b) Equivalent NFA for the language of 2(a) (13) · Q4(b) Convert the ε-NFA (q0…q10 figure) to a DFA, showing computations (20)
2019-20	Q3(b) Minimal NFA for {a^n b^n | n ≤ 3} (15) · Q4(a) NFA with 5 states → DFA: minimum and maximum number of states (5)
2017-18	Q3(a) NFA: two 0s with a nonzero multiple-of-4 number of characters between; diagram + table (18) · Q4(a) ε-NFA (p, q, r table): strings of length ≤ 3 accepted; convert to DFA (15)
2016-17	Q2(b) Convert the ε-NFA (p, q, r, s figure) to a DFA (8) · Q3(a) NFAs: final digit appeared before; final digit not appeared before; two 0s separated by a multiple of 4 positions (20)
2014-15	Q3(a) NFA: nonzero length strings with a nonzero even number of a's at the end (13⅓) · Q4(a) NFA for an even number of occurrences of 01; convert to DFA (reachable part) (11⅓)
2013-14	Q7(a) Eliminate all ε-transitions from the given NFA (q0…q4) (9) · Q8(b) Prove every NFA has an equivalent DFA (10⅓)
2012-13	Q8(a) Three differences between DFA and NFA with examples (5)
2011-12	Q5(a) Three differences between DFA and NFA with examples (6) · Q7(b) Convert the NFA of 7(a) to a DFA (reachable states only) (8)
2010-11	Q1 Will the DFA built from an NFA by subset construction accept the same language? Justify (10) · Q3 Convert the NFA (p,q,r,s∗,t∗ table) to a DFA using lazy evaluation (10)
2009-10	Q1 'Nondeterminism is an inessential feature of finite automata' — explain (10) · Q3 Convert the ε-NFA (q1…q5 figure) to a DFA using lazy evaluation (10)
2008-09	Q5(c) Define a DFA and an NFA (5⅓) · Q6(b) Example of an NFA with n+1 states that has no equivalent DFA with fewer than 2^n states (9) · Q6(c) ε-NFA (p,q,r table): ε-closure of each state; convert to DFA (5⅓)
2007-08	Q2(a) Prove L(D) = L(N) for the DFA D obtained from NFA N by subset construction (10) · Q3(c) 'Nondeterminism is an inessential feature of finite automata' — explain (5⅓)
2006-07	Q2(a) Prove: if D is the DFA built from NFA N by subset construction then L(D) = L(N) (10) · Q2(c) 'Nondeterministic computation is a tree of possibilities' — elaborate with example (5⅓) · Q3(b) Steps to find the equivalent DFA of an ε-NFA using lazy evaluation (8)

EXAM FOCUS Two question types dominate. (1) **Convert an (ε-)NFA to a DFA** — 2021-22, Jan-2020 (twice), 2017-18, 2016-17, 2014-15, 2011-12, 2010-11, 2009-10, 2008-09: learn the table method of Lesson 9 until it is mechanical; the same p/q/r ε-NFA table was set in 2008-09, 2017-18 and Jan-2020. (2) **Design an NFA** — every paper since 2014-15 (10–20 marks): 'guess where it starts', 'guess the split', union with ε. Theory parts (DFA vs NFA, why the equivalence is surprising, 2^n blow-up, the L(D) = L(N) proof) are 4–10 marks each.

Lesson 6 — Nondeterminism and the NFA

Slides 1–49 (Lec04 §4A, §4B)

1. Find versus Verify (sl. 4–20)

In deterministic computation every step is forced. Many tasks have a hard *find* version and an easy *verify* version: maze (find a path / check a path), Sudoku, password, SAT, proofs. A nondeterministic machine is allowed to **guess** the right branch; if the guess can be verified, it accepts. Three equivalent views for finite automata (sl. 15–16): magical guessing, parallel universes (all choices explored at once), and a computation tree. It is an idealised model, like infinite tapes, but it models the shape of search: backtracking, SAT solvers, theorem provers, regex engines, AI planning (sl. 17–20).

2. A First NFA: Strings Containing 101 (sl. 22–33)

A DFA tracks partial progress (nothing, 1, 10, 101). An NFA instead guesses, at some 1, 'this 1 starts 101': q0 loops on 0, 1; q0 −1→ q1 −0→ q2 −1→ q3, q3 loops (accept). δ(q0, 1) = {q0, q1}: ignore this 1, or start the pattern.

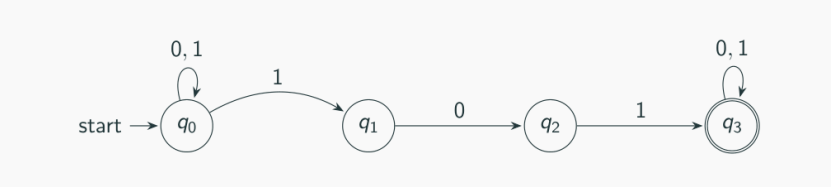


Figure 6.1 — NFA for strings containing 101 (sl. 29)

Branch outcome	Meaning
Accepted	input fully read and the branch ends in an accepting state
Rejected	input fully read, branch ends in a non-accepting state
Dead	the branch gets stuck (no move) before the input is fully read

The NFA accepts if at least one branch is accepted (sl. 36–37). To prove acceptance show one accepting branch; to prove rejection you must rule out every branch (sl. 38–40). 'A DFA has one timeline; an NFA has many' (sl. 41).

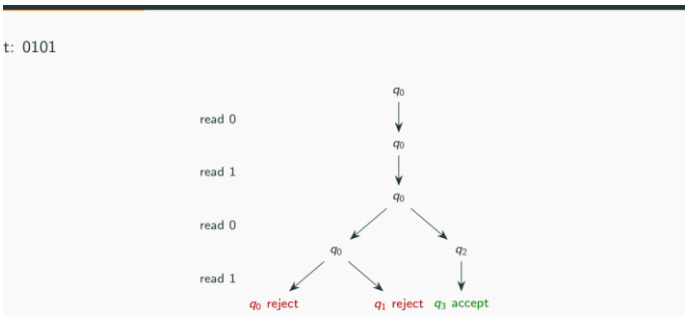


Figure 6.2 — Computation tree for 0101: one leaf reaches q3, so 0101 is accepted (sl. 34)

For 1000 (sl. 35) the guessed branch q0→q1→q2 dies on the second 0, and the surviving branch ends in q0 — rejected.

3. Formal Definition (sl. 42–49)

$$N = (Q, \Sigma, \delta, q_0, F)$$
$$\delta : Q \times \Sigma \to P(Q) \quad (\varepsilon\text{-NFA: } \delta : Q \times \Sigma\varepsilon \to P(Q))$$

Only δ differs from a DFA: it returns a **set** of states (an element of the power set P(Q)). δ(q, a) = ∅ — no branch survives; {r} — one possibility; {r, s, t} — three branches (sl. 45). Table for the 101-NFA (sl. 46):

```
        0       1
→ q0   {q0}    {q0,q1}
  q1   {q2}    ∅
  q2   ∅       {q3}
* q3   {q3}    {q3}        (sl. 46: NFA for 'contains 101')
```

Every DFA is an NFA: write δ(q, a) = r as δ(q, a) = {r} (sl. 48–49). The big question is the converse — answered in Lesson 9.

4. DFA versus NFA

	DFA	NFA
Transition function	δ : Q × Σ → Q (one state)	δ : Q × Σε → P(Q) (a set, possibly ∅)
Moves per (state, symbol)	exactly one	zero, one or many
ε-moves	not allowed	allowed (Sipser's NFA)
Computation	a single path	a tree of paths; accept if some branch accepts
Missing transitions	not allowed (use a dead state)	allowed — the branch dies
Design	must track all information explicitly	can guess (often far fewer states)
Power	regular languages	the same regular languages
Example: 3rd symbol from end is 1	8 states	4 states

Exam Approach

1. Definitions: write the 5-tuple and say exactly how δ differs.
2. 'Tree of possibilities': draw the tree for one short input, mark accepted / rejected / dead leaves.
3. Differences: give three rows of the table above, each with a one-line example.

Practice — Lesson 6

(a) Actual — 2011-12 Q5(a) / 2012-13 Q8(a). (b) Actual — 2008-09 Q5(c). (c) Actual — 2006-07 Q2(c).

(a) Give three differences between DFA and NFA with examples. (6)

(b) Define a deterministic finite automaton and a nondeterministic finite automaton. (5⅓)

(c) 'One way to think of a nondeterministic computation is as a tree of possibilities' — elaborate with an example. (5⅓)

✓ Solution

(a) (1) δ: a DFA has exactly one next state for each (state, symbol); an NFA may have several or none — e.g. in the 101-NFA δ(q0, 1) = {q0, q1} and δ(q1, 1) = ∅. (2) ε-moves: an NFA may change state without reading input (q −ε→ r); a DFA cannot. (3) Acceptance: a DFA follows one path; an NFA accepts if *any* branch ends in an accepting state — so 0101 is accepted by the 101-NFA although the branch that stays in q0 rejects. (Also: NFAs are often smaller — 3rd-from-end needs 4 NFA states vs 8 DFA states — but both recognise exactly the regular languages.)

(b) DFA: M = (Q, Σ, δ, q0, F), δ : Q × Σ → Q, accepts w iff the unique run ends in F. NFA: N = (Q, Σ, δ, q0, F), δ : Q × Σε → P(Q); N accepts w iff some sequence of choices (with ε-moves) consumes w and ends in F.

(c) Each node of the tree is a state reached after some prefix; a node branches when δ gives several states, and a branch ends when δ gives ∅. The NFA accepts iff some leaf at depth |w| is accepting.

Example (sl. 34): the tree for 0101 in the 101-NFA has leaves q0 (reject), q1 (reject) and q3 (accept), so 0101 is accepted. The machine can be thought of as exploring all branches in parallel; a DFA simulation keeps the *set* of nodes at each level — exactly the subset construction.

Lesson 7 — Designing NFAs

Slides 50–70 (Lec04 §4C) + TF design problems

1. Three Guessing Patterns (sl. 51–70)

Pattern	Slide example	How the NFA guesses
Where a pattern begins	contains fed or dff over {d,e,f} (sl. 51–55)	q0 loops on everything; on f guess 'fed starts', on d guess 'dff starts'
Where a split point is	w1w2: |w1| ≡ 0 mod 3, w2 has even #b (sl. 56–62)	count |w1| mod 3; at r0 jump into the parity part while reading the next symbol
Which condition holds	missing at least one of a, b, c (sl. 63–68)	on the first symbol branch to q¬a, q¬b or q¬c; a branch dies if it reads its missing symbol

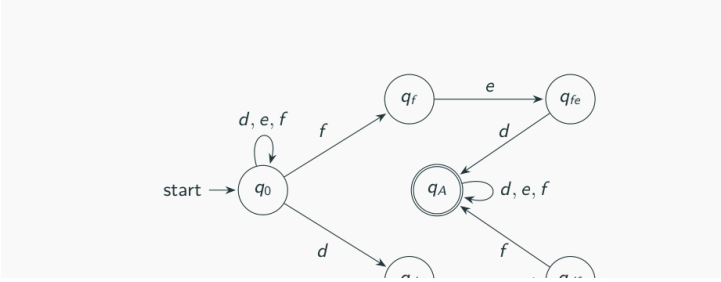


Figure 7.1 — NFA for 'contains fed or dff' (sl. 55)

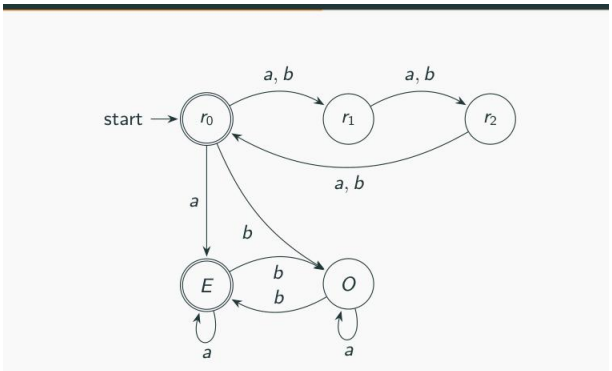


Figure 7.2 — Guessing the split point without ε (sl. 60)

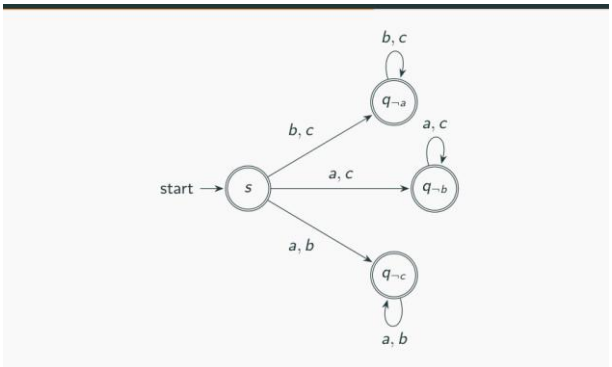


Figure 7.3 — Guess which symbol is missing (sl. 67)

MEMORIZE An NFA is good at expressing 'there exists a good choice' (sl. 69–70): a substring somewhere, a split point somewhere, one of several conditions. Union = new start state with ε to each machine. 'Some position from the end' = loop, then a guessed chain.

Exam Approach — NFA Design

1. Find the 'exists' in the language (some occurrence, some split, one of the cases).
2. Let a start (or loop) state read anything, then guess the moment with a branch.
3. Verify the guessed part with a short deterministic chain; let wrong guesses die (omit transitions).
4. Union of conditions: separate machines joined by ε from a new start.
5. Check: one accepted string (show the branch) and one rejected string (all branches fail).

TRAP Make sure a guess cannot be taken at the wrong moment. For 'even number of a's at the END', the final block must be maximal: start it only at the beginning or right after a b, otherwise aaa would be accepted (guess after the first a).

Practice — Lesson 7

(a) Actual — 2014-15 Q3(a). (b) Actual — 2017-18 Q3(a) / 2016-17 Q3(a)(iii). (c) Actual — 2016-17 Q3(a)(i)–(ii). (d) Actual — 2021-22 Q3(b). (e) Actual — 2022-23 Q4(b).

(a) Design an NFA over {a, b} accepting all and only the strings of nonzero length having a nonzero even number of a's at the end (accept aa, aaaa, abbaa, abaababaaaa; reject aaa, ab, b, abbaaa, abaabbbb). (13⅓)

(b) NFA for strings over {0,1} with at least two 0s separated by a nonzero number of characters that is a multiple of 4 (accept 00000, 101111010, 01000010111011; reject 00, 10101010); diagram and table. (18) — and the 2016-17 version where 0 characters between is also allowed (00 accepted). (7)

(c) NFAs over {0, …, 9}: (i) the final digit has appeared before; (ii) the final digit has not appeared before. (5 + 8)

(d) NFA that accepts binary strings containing at least two 0s, or exactly two 1s. (10)

(e) NFA accepting the binary strings that begin and end with different symbols and have odd length. (15)

✓ Solution

(a) Idea: guess where the final block of a's starts; it must start at the beginning or right after a b. L = loop state; B = 'a b was just read'; A1/A2 count the final a's mod 2.

```
      a         b
→  s   {A1,L}    {B,L}
   A1  {A2}      Ø
 * A2  {A1}      Ø
   B   {A1}      Ø
   L   {L}       {B,L}
(→ start, * accepting; Ø = no move — that branch dies)
```

abbaa: s −a→ L −b→ L −b→ B −a→ A1 −a→ A2 ✓ accept. aaa: the block must start at position 1 (s −a→ A1 → A2 → A1) — ends in A1, reject; any later start is impossible because no b precedes it ✓.

(b) After the first chosen 0 count the characters read mod 4; a second 0 is allowed when the count is a nonzero multiple of 4 (e4). F then loops.

```
      0         1
→  s   {d0,s}    {s}
 * F   {F}       {F}
   d0  {e1}      {e1}
   e1  {e2}      {e2}
   e2  {e3}      {e3}
   e3  {e4}      {e4}
   e4  {F,e1}    {e1}
(→ start, * accepting; Ø = no move — that branch dies, e1…e4 = 1…4 characters between (mod 4, e4 =
4, 8, …))
```

101111010: zeros at positions 2 and 7 have 4 characters between — branch s −1→ s −0→ d0 −1→ e1 −1→ e2 −1→ e3 −1→ e4 −0→ F −1→ F ✓. 10101010: every gap between zeros is 1, 3 or 5 characters — all branches fail ✓.

> **NOTE** The printed example 00000 does not fit any reading together with the other four: its gaps are 0–3 characters. With 'number of characters between is a nonzero multiple of 4', the other four examples are classified exactly as printed; 00000 is probably a misprint of 000000 (gap of 4 between the 1st and 6th zero). Say this in one line and design for the stated rule.

2016-17 version ('0 is an allowable multiple', 00 accepted): there the first example shows that adjacent 0s count, i.e. the number of characters between is a multiple of 4 (0, 4, 8, …):

```
      0         1
→  s   {c0,s}    {s}
 * F   {F}       {F}
   c0  {F,c1}    {c1}
   c1  {c2}      {c2}
   c2  {c3}      {c3}
   c3  {c0}      {c0}
(→ start, * accepting; ∅ = no move — that branch dies, c0…c3 = characters between mod 4)
```

In 2016-17 the examples 10111010 and 01010111011 fit the other reading, 'positions differ by a multiple of 4' (zeros at positions 2, 6 and 1, 5, 9), while 00 fits only 'characters between = 0'. Write which reading you use; for the distance reading (j − i a positive multiple of 4):

```
      0         1
→  s   {s,t0}    {s}
 * F   {F}       {F}
   t0  {t1}      {t1}
   t1  {t2}      {t2}
   t2  {t3}      {t3}
   t3  {F,t0}    {t0}
(→ start, * accepting; ∅ = no move — that branch dies, t0 = just read the first chosen 0; t3 −0→ F
completes a distance of 4, 8, …)
```

(c) (i) Guess the final digit d at an earlier occurrence, keep reading, and finish on d. (ii) Guess the final digit d at the start and read only digits ≠ d until the final d. Shown for digits {0,1,2}; for {0,…,9} repeat the pattern for every digit (21 and 21 states).

```
      0         1         2
→  s   {p0,s}    {p1,s}    {p2,s}
 * f   ∅         ∅         ∅
   p0  {f,p0}    {p0}      {p0}
   p1  {p1}      {f,p1}    {p1}
   p2  {p2}      {p2}      {f,p2}
(→ start, * accepting; ∅ = no move — that branch dies, (i))
```

```
      0            1            2            ε
→  s   ∅             ∅            ∅            {r0,r1,r2}
 * f   ∅             ∅            ∅            ∅
   r0  {f}           {r0}         {r0}         ∅
   r1  {r1}          {f}          {r1}         ∅
   r2  {r2}          {r2}         {f}          ∅
(→ start, * accepting; ∅ = no move — that branch dies, (ii))
```

(d) Union of two NFAs: N1 'at least two 0s' (s1 → t1 → u1 on 0s, loops elsewhere) and N2 'exactly two 1s' = 0*10*10*. A new start s has ε-moves to both.

```
      0          1          ε
→  s   ∅          ∅          {s1,s2}
   s1  {s1,t1}    {s1}       ∅
   s2  {s2}       {t2}       ∅
   t1  {t1,u1}    {t1}       ∅
   t2  {t2}       {u2}       ∅
 * u1  {u1}       {u1}       ∅
 * u2  {u2}       ∅          ∅
(→ start, * accepting; ∅ = no move — that branch dies)
```

(e) Branch on the first symbol, track length parity, and allow the final symbol (the other symbol) only when it makes the length odd.

```
      0         1
→  s   {x0}      {x1}
 * F   ∅         ∅
   x0  {y0}      {y0}
   x1  {y1}      {y1}
   y0  {x0}      {F,x0}
   y1  {F,x1}    {x1}
(→ start, * accepting; ∅ = no move — that branch dies, x = odd length so far, y = even; y –(other
symbol)→ F)
```

011: s −0→ x0 −1→ y0 −1→ F ✓ (length 3, starts 0, ends 1). 0101: length even, every branch ends in x0/y0 — reject ✓.

Practice — Lesson 7 (continued)

(f) Actual — 2019-20 Q3(b). (g) Actual — 2023-24 Q3(a). (h) Actual — 2023-24 Q4(a). (i) Actual — Jan-2020 Q2(b).

(f) Design an NFA for L = {a^n b^n | n ≤ 3} over {a, b}. You may be penalised for using more states than the required minimum. (15)

(g) Find an NFA with three states for L = {a^n | n ≥ 1} ∪ {b^m a^k | m ≥ 0, k ≥ 0}. Can the language be accepted by an NFA with fewer than three states? (15)

(h) Find an NFA that accepts {a}* such that if a single edge is removed (nothing else changed) the automaton accepts {a}. (10)

(i) For the DFA of Jan-2020 Q2(a) (strings whose last two symbols are equal), draw an equivalent NFA. (13)

✓ Solution

(f) L = {ε, ab, aabb, aaabbb}. Count the a's (q0 … q3) and share the b-chain so that each count needs exactly as many b's: q1 −b→ f, q2 −b→ p1 −b→ f, q3 −b→ p2 −b→ p1 −b→ f. q0 is accepting (n = 0).

```
      a       b
→* q0  {q1}    ∅
 * f   ∅       ∅
   p1  ∅       {f}
   p2  ∅       {p1}
   q1  {q2}    {f}
   q2  {q3}    {p1}
   q3  ∅       {p2}
(→ start, * accepting; ∅ = no move — that branch dies)
```

Minimality (fooling set): the 7 pairs (ε, aaabbb), (a, aabbb), (aa, abbb), (aaa, bbb), (aaab, bb), (aaabb, b), (aaabbb, ε) all concatenate to a member of L, but for any two pairs (x, y), (x′, y′) at least one of xy′, x′y is not in L. Every pair needs its own state on an accepting path, so any NFA needs ≥ 7 states — the design is minimal (checked by a program).

(g) Note first that a^n (n ≥ 1) is already b^0a^n, so L = b*a* (including ε). A 3-state NFA: s (start), B (reading b's), A (reading a's), all accepting:

```
      a      b
→* s   {A}    {B}
 * A   {A}    Ø
 * B   {A}    {B}
(→ start, * accepting; Ø = no move — that branch dies)
```

Fewer states: **yes, 2 states suffice**: q0 (start, accepting, loop b), q0 −a→ q1 (accepting, loop a); missing transitions (q1 on b) kill the branch.

```
      a       b
→* q0  {q1}    {q0}
 * q1  {q1}    Ø
(→ start, * accepting; Ø = no move — that branch dies)
```

One state is impossible: a 1-state NFA accepts ∅, {ε}, or S* where S is the set of looped symbols — a*, b* or (a ∪ b)* — none of which equals b*a* (ba ∉ L but would be accepted by (a ∪ b)*). So the minimum is 2.

(h) S (start, non-accepting) −ε→ M (accepting, loop a), and S −a→ F (accepting). L = a* ∪ {a} = a*. Remove the edge S −ε→ M: M becomes unreachable and only S −a→ F remains, so L = {a}.

```
      a      ε
→  S   {F}    {M}
 * F   Ø      Ø
 * M   {M}    Ø
(→ start, * accepting; Ø = no move — that branch dies)
```

(i) Guess the last two symbols: s loops on 0, 1; s −0→ a −0→ F, s −1→ b −1→ F.

```
      0        1
→  s   {a,s}    {b,s}
 * F   Ø        Ø
   a   {F}      Ø
   b   Ø        {F}
(→ start, * accepting; Ø = no move — that branch dies)
```

Lesson 8 — ε-Transitions and Removing Them

Slides 71–90 (Lec04 §4D)

1. ε-NFA and Acceptance (sl. 72–78)

An ε-NFA may move without reading input: δ : Q × Σε → P(Q), Σε = Σ ∪ {ε}. It accepts w if there is a path from q0 to an accepting state whose labels, after deleting the ε's, spell exactly w (sl. 73–74). ε-moves make automata modular: for {ab, acd}, read the common prefix a, then jump freely into the b-module or the cd-module (sl. 75–78).

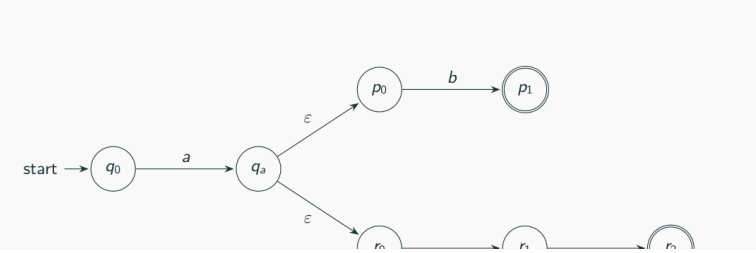


Figure 8.1 — ε-NFA for {ab, acd} (sl. 76)

2. ε-Closure (sl. 79)

E(q) = { r : r reachable from q using zero or more ε-moves } (q ∈ E(q))

E(S) = ∪ E(q) for q ∈ S

3. Removing ε-Transitions (sl. 80–90)

δ′(q, a) = E(∪ δ(r, a) for r ∈ E(q))

F′ = { q : E(q) ∩ F ≠ ∅ }

'Take free moves, read a, then take free moves again.' A state that can reach an accepting state by ε-moves becomes accepting. Worked example (sl. 83–87): Q = {q0, …, q4}, F = {q4}, q0 −ε→ q1, q1 −0→ q2, q1 −1→ q3, q2 −ε→ q4, q3 −ε→ q1, q3 −1→ q4.

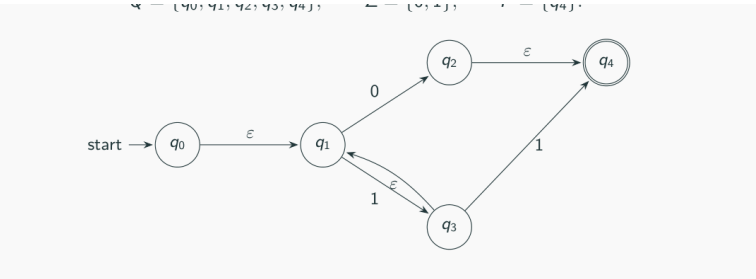


Figure 8.2 — The slide's ε-NFA over {0, 1} (sl. 83)

```
Step 1  ε-closures:   E(q0) = {q0,q1}   E(q1) = {q1}   E(q2) = {q2,q4}   E(q3) = {q1,q3}   E(q4) =
{q4}
Step 2  F′ = {q2,q4}   (q2 reaches q4 by ε)
Step 3  δ′:      0            1
        q0     {q2,q4}      {q1,q3}
        q1     {q2,q4}      {q1,q3}
        q2     ∅            ∅
        q3     {q2,q4}      {q1,q3,q4}
        q4     ∅            ∅
e.g. δ′(q0,0): E(q0) = {q0,q1}; δ(q0,0) ∪ δ(q1,0) = {q2}; E({q2}) = {q2,q4}
```

ε-transitions add no power: ε-NFA ⇔ NFA without ε (sl. 88–89). Following Sipser, from now on 'NFA' allows ε-moves (sl. 90).

Exam Approach — Eliminate ε

1. Write E(q) for every state.
2. F′ = states whose closure contains an accepting state.
3. For each q and a: union δ over E(q), then close again.
4. Draw the new NFA (same states, no ε); drop unreachable states.

TRAP Close twice: before reading the symbol (E(q)) and after (E of the result). Forgetting the second closure is the commonest mistake; forgetting F′ loses strings accepted after trailing ε-moves.

Practice — Lesson 8

(a) Actual — 2013-14 Q7(a). (b) Actual — 2008-09 Q6(c)(i).

(a) Eliminate all ε-transitions from the NFA: start q0, accepting q2; q0 −a→ q1, q0 −ε→ q4, q1 −ε→ q2, q2 −a→ q1, q4 −b→ q1, q4 −a→ q3, q3 −b→ q1, q3 −ε→ q2. (9)

(b) For the ε-NFA →p: a→{p}, b→{q}, c→{r}, ε→∅; q: a→{q}, b→{r}, ε→{p}; *r: a→{r}, c→{p}, ε→{q}, compute the ε-closure of each state. (2008-09 version of the table)

✓ Solution

(a) Given NFA and the computation:

```
      a       b       ε
→  q0  {q1}    ∅       {q4}
   q1  ∅       ∅       {q2}
 * q2  {q1}    ∅       ∅
   q3  ∅       {q1}    {q2}
   q4  {q3}    {q1}    ∅
(→ start, * accepting; ∅ = no move — that branch dies)
```

```
ε-closures: E(q0) = {q0,q4}   E(q1) = {q1,q2}   E(q2) = {q2}   E(q3) = {q2,q3}   E(q4) = {q4}
F′ = {q1,q2,q3}   (E(q1), E(q3) contain q2; q0's closure {q0,q4} does not)
δ′        a              b
  q0      {q1,q2,q3}     {q1,q2}
  q1      {q1,q2}        ∅
  q2      {q1,q2}        ∅
  q3      {q1,q2}        {q1,q2}
  q4      {q2,q3}        {q1,q2}
```

Example entry: δ′(q0, a) = E(δ(q0,a) ∪ δ(q4,a)) = E({q1, q3}) = {q1, q2, q3}. The resulting NFA (no ε) accepts exactly the same strings — checked on all strings up to length 8.

(b) E(p) = {p} E(q) = {p,q} E(r) = {p,q,r}. (r reaches q by ε, and q reaches p.)

Lesson 9 — NFA → DFA: the Subset Construction

Slides 91–109 (Lec04 §4E) · ε-NFA version and proof follow the textbook

1. Idea and Formal Construction (sl. 92–96)

Two machines are equivalent iff they recognise the same language. Every DFA is an NFA; the surprise is that **every NFA has an equivalent DFA**, so nondeterminism adds no power to finite automata. A DFA cannot be in several states at once, so let each DFA state be a *set* of NFA states — all the states the NFA could be in.

$$Q' = P(Q) \quad q0' = \{q0\} \quad F' = \{ R \in Q' : R \cap F \neq \emptyset \}$$
$$\delta'(R, a) = \cup\, \delta(r, a) \text{ for } r \in R$$

With ε-moves (Sipser, textbook): start = E({q0}) and δ′(R, a) = E(∪ δ(r, a)). If |Q| = n the DFA has at most 2^n states; build only the reachable ones (**lazy evaluation**).

2. Worked Example: Contains aba (sl. 97–108)

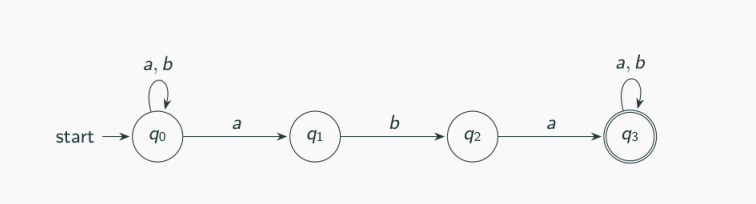


Figure 9.1 — NFA for strings containing aba (sl. 98)

	DFA state	NFA set	a	b
→	A	{q0}	B	A
	B	{q0,q1}	B	C
	C	{q0,q2}	D	A
*	D	{q0,q1,q3}	D	E
*	E	{q0,q2,q3}	D	F
*	F	{q0,q3}	D	F

(→ start, * accepting = contains an NFA accepting state, A–F as on sl. 99–107)

Only 6 of 16 subsets are reachable. The DFA tracks the longest matching prefix of aba (sl. 108). There may be smaller equivalent DFAs than the construction gives (sl. 109) — here D, E, F could be merged into one accepting sink.

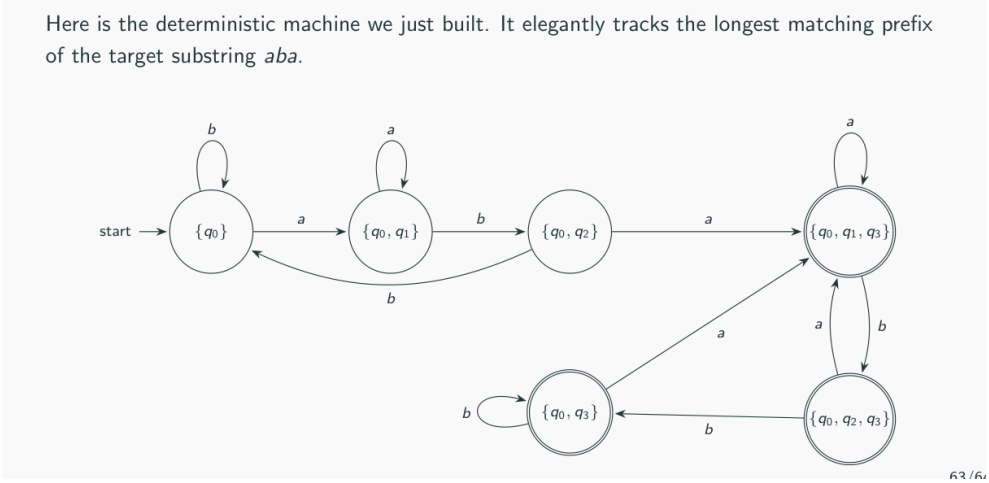


Figure 9.2 — The resulting DFA (sl. 108)

3. Lazy Evaluation — the Exam Procedure (2006-07 Q3(b))

1. (ε-NFA) Compute E(q) for every state.
2. Start state A = E({q0}). Put it in a queue.
3. Take a set S from the queue; for each symbol a compute T = E(∪ δ(r, a), r ∈ S). If T is new, give it a letter and add it to the queue. Record S −a→ T (T may be ∅ — a dead state).
4. Repeat until the queue is empty. Accepting DFA states = sets containing an NFA accepting state.
5. Draw the DFA / write its table. Only reachable subsets ever appear.

4. Proof that L(D) = L(N) (2006-07, 2007-08, 2010-11, 2013-14)

EXAM-READY ANSWER Claim: for every string w, $\hat{\delta}_D(\{q0\}, w) = \hat{\delta}_N(q0, w)$, the set of states N can reach on w. Induction on |w|. Basis w = ε: both sides are {q0}. Step w = xa: assume $\hat{\delta}_D(\{q0\}, x) = \hat{\delta}_N(q0, x) = \{p1, \ldots, pk\}$. Then $\hat{\delta}_D(\{q0\}, xa) = \delta_D(\{p1, \ldots, pk\}, a) = \cup\, \delta_N(pi, a)$ (definition of δ_D) $= \hat{\delta}_N(q0, xa)$ (definition of $\hat{\delta}_N$). Hence D accepts w ⇔ $\hat{\delta}_D(\{q0\}, w) \in F_D \Leftrightarrow \hat{\delta}_N(q0, w)$ contains a state of F_N ⇔ N accepts w. So L(D) = L(N). With ε-moves use E everywhere and the same induction. Therefore a language is accepted by some DFA iff by some NFA.

5. How Big Can the DFA Be? (2008-09 Q6(b), 2019-20 Q4(a))

L_n = {w : the n-th symbol from the right is 1}. NFA with n + 1 states: q0 loops on 0, 1; q0 −1→ q1 −0,1→ q2 … −0,1→ qn (accept). Any DFA needs at least 2^n states: take two different strings u ≠ v of length n; they differ at some position i from the left (say u has 1 there, v has 0). Append the same i − 1 symbols to both: position i becomes the n-th symbol from the end, so u·z ∈ L_n but v·z ∉ L_n. Hence the 2^n strings of length n must all lead to different DFA states.

```
      0          1
→  q0  {q0}       {q0,q1}
   q1  {q2}       {q2}
   q2  {q3}       {q3}
 * q3  ∅          ∅
(→ start, * accepting; ∅ = no move — that branch dies, n = 3: 4 NFA states; its DFA needs 2^3 = 8
states)
```

MEMORIZE NFA with n states → DFA with at least 1 and at most 2^n states (one per subset, ∅ included). For n = 5: minimum 1 (e.g. the NFA accepts Σ* or ∅), maximum 2^5 = 32. The upper bound is reached by known families of NFAs; 'nth symbol from the end' already forces 2^(n−1) for n states.

6. Why the Equivalence Is Surprising and Useful (2021-22 Q2(a); 2007-08 Q3(c); 2009-10 Q1)

EXAM-READY ANSWER Surprising: an NFA seems more powerful — it can guess and explore exponentially many paths in parallel — yet for finite automata guessing adds no power: a DFA can simulate it by remembering the set of possible states (finitely many sets). Useful: we may *design* with NFAs (short, modular, easy closure proofs for ∪, ∘, *; RE → NFA is easy) and *implement* with DFAs (one pass, one state, O(1) per symbol), converting mechanically. That is why 'nondeterminism is an inessential feature of finite automata': it changes size and convenience, never the class of languages (the price can be an exponential number of states).

Exam Approach — NFA → DFA

1. Write the ε-closures first (even if the question does not ask).
2. Lazy evaluation with a table: DFA state | NFA set | one column per symbol.

3. Mark start (E({q0})) and accepting sets; include ∅ as the dead state if it appears.
4. Draw the DFA. If asked, describe the language and check one string on both machines.

TRAP With ε-moves the DFA start state is E({q0}), not {q0}. A set is accepting if it contains *any* NFA accepting state. Do not forget transitions out of ∅ (∅ −a→ ∅).

Practice — Lesson 9

(a) Actual — 2017-18 Q4(a), Jan-2020 Q1(c), 2008-09 Q6(c). (b) Actual — 2021-22 Q3(c). (c) Actual — 2016-17 Q2(b). (d) Actual — Jan-2020 Q4(b).

(a) For the ε-NFA (→p: a→{p}, b→{q}, c→{r}; q: a→{q}, b→{r}, ε→{p}; *r: a→{r}, c→{p}, ε→{q}): (i) give, with explanations, the general forms of all strings of length three or less accepted; (ii) convert it to a DFA. (15)

(b) Convert to a DFA: q1: a→{q3}, ε→{q2}; q2: a→{q1}; q3: a→{q2}, b→{q2, q3}. (9)

(c) Convert the NFA of 2016-17 Q2(b) to a DFA: p −0→ q, p −ε→ r, q −0→ q, q −ε→ r, r −1→ q, r −ε→ s, s −0→ p, s −0→ r; start p, accepting q. (8)

(d) Convert the ε-NFA of Jan-2020 Q4(b) (q0 … q10, the Thompson NFA of (0 ∪ 1)*011) to a DFA, showing the computations. (20)

✓ Solution

(a) ε-closures: E(p) = {p}, E(q) = {p,q}, E(r) = {p,q,r}.

```
   DFA state NFA set  a       b       c
→  A         {p}      A       B       C
   B         {p,q}    B       C       C
 * C         {p,q,r}  C       C       C
(→ start, * accepting = contains an NFA accepting state)
```

Meaning: A = only a's read so far (still {p}); B = exactly one b and no c; C = a c or a second b has been read — C is accepting and absorbing. So L = strings over {a, b, c} that contain at least one c or at least two b's. Length ≤ 3 (30 of the 40 strings): every string containing c (c, ac, bc, ca, cb, cc, and all length-3 strings with a c), plus bb, abb, bab, bba, bbb. Rejected: ε, a, b, aa, ab, ba, aaa, aab, aba, baa.

(b) The table marks no start or accepting state; take q1 as start (first row) and q3 as accepting (state your assumption — the subset construction itself does not depend on it).

```
   DFA state NFA set     a       b
→  A         {q1,q2}     B       C
 * B         {q1,q2,q3}  B       D
   C         ∅           C       C
 * D         {q2,q3}     A       D
(→ start, * accepting = contains an NFA accepting state, accepting = sets containing q3)
```

(c) ε-closures: E(p) = {p, r, s}, E(q) = {q, r, s}, E(r) = {r, s}, E(s) = {s}.

```
   DFA state NFA set    0       1
→  A         {p,r,s}    B       C
 * B         {p,q,r,s}  B       C
 * C         {q,r,s}    B       C
(→ start, * accepting = contains an NFA accepting state)
```

A = E(p) does not contain q, so ε is rejected; both symbols lead from A to accepting sets, and B, C never return to A. Hence L = $(0 \cup 1)^+$ — every non-empty string (checked by program).

(d) Closures used: E(q0) = {q0,q1,q2,q4,q7}; after reading 0 from q2 or 1 from q4 we reach q3/q5, whose closures add q6, q1, q2, q4, q7.

```
   DFA state NFA set                  0       1
→  A         {q0,q1,q2,q4,q7}         B       C
   B         {q1,q2,q3,q4,q6,q7,q8}   B       D
   C         {q1,q2,q4,q5,q6,q7}      B       C
   D         {q1,q2,q4,q5,q6,q7,q9}   B       E
 * E         {q1,q2,q4,q5,q6,q7,q10}  B       C
(→ start, * accepting = contains an NFA accepting state)
```

The DFA tracks the progress toward the suffix 011 (A/C: no progress, B: 0, D: 01, E: 011). A and C are equivalent, so the minimal DFA has 4 states. L = (0 ∪ 1)*011 — checked by program.

Practice — Lesson 9 (continued)

(e) Actual — 2010-11 Q3. (f) Actual — 2009-10 Q3. (g) Actual — 2011-12 Q7(a)–(b). (h) Actual — 2014-15 Q4(a).

(e) Convert using lazy evaluation: →p: 0→{p,q}, 1→{p}; q: 0→{r,s}, 1→{t}; r: 0→{p,r}, 1→{t}; *s, *t: ∅. (10)

(f) Convert the ε-NFA of 2009-10 Q3 (q1 −ε→ q2; q2 −a→ q3, q2 −b→ q4; q3 −a→ q2, q3 −b→ q3, q3 −ε→ q5; q4 −b→ q2, q4 −a→ q3, q4 −ε→ q5; start q1, accept q5) to a DFA using lazy evaluation. (10)

(g) Construct an NFA for (01 ∪ 001 ∪ 010)*, then convert it to a DFA with only the states reachable from the start. (8 + 8)

(h) NFA for strings over {0,1} with an even number of occurrences of the substring 01; convert to a DFA (reachable portion only). (7⅓ + 4)

✓ Solution

(e)

```
   DFA state NFA set    0       1
→  A         {p}        B       A
   B         {p,q}      C       D
 * C         {p,q,r,s}  C       D
 * D         {p,t}      B       A
(→ start, * accepting = contains an NFA accepting state)
```

(f) E(q1) = {q1, q2}, E(q3) = {q3, q5}, E(q4) = {q4, q5}; others are their own closures.

```
   DFA state NFA set  a       b
→  A         {q1,q2}  B       C
 * B         {q3,q5}  D       B
 * C         {q4,q5}  B       D
   D         {q2}     B       C
(→ start, * accepting = contains an NFA accepting state)
```

(g) NFA: from s (start, accepting) read one of the blocks and return. 01 and 010 share the prefix 01 and 001 shares the first 0, so: s −0→ p; p −1→ s (block 01) or p −1→ r −0→ s (block 010); p −0→ u −1→ s (block 001).

```
      0       1
→* s   {p}     ∅
   p   {u}     {r,s}
   r   {s}     ∅
   u   ∅       {s}
(→ start, * accepting; ∅ = no move — that branch dies)
```

```
   DFA state NFA set  0       1
→* A         {s}      B       C
   B         {p}      D       E
   C         ∅        C       C
   D         {u}      C       A
 * E         {r,s}    F       C
 * F         {p,s}    G       E
   G         {p,u}    D       E
(→ start, * accepting = contains an NFA accepting state, C = ∅ (dead))
```

(h) Every binary string is 1*(0⁺1⁺)^k 0*, and k = number of 01's. Even k ⇔ 1*(0⁺1⁺0⁺1⁺)*0*. NFA from this expression (ε-moves at m and from d back to m):

```
      0      1      ε
→  s   Ø      {s}    {m}
   a   {a}    {b}    Ø
   b   {c}    {b}    Ø
   c   {c}    {d}    Ø
   d   Ø      {d}    {m}
 * e   {e}    Ø      Ø
   m   {a}    Ø      {e}
(→ start, * accepting; Ø = no move — that branch dies)
```

```
   DFA state NFA set  0       1
→* A         {e,m,s}  B       A
 * B         {a,e}    B       C
   C         {b}      D       C
   D         {c}      D       E
 * E         {d,e,m}  B       E
(→ start, * accepting = contains an NFA accepting state)
```

A and E are equivalent (both accepting, same moves), so the minimal DFA has 4 states: (even, last not 0), (even, last 0), (odd, last 0), (odd, last 1).

Quick Notes — NFA

Revision points for Lessons 6–9.

L6 · Nondeterminism

- Find vs verify; NFA guesses, accepts if SOME branch accepts.
- δ : Q × Σε → P(Q); ∅ = branch dies. Every DFA is an NFA.
- Accept: show one branch. Reject: rule out all branches.
- Differences: δ set-valued, ε-moves, missing moves allowed, tree vs path; same power.

L7 · Design

- Guess where a pattern starts / where the split is / which case holds.
- Union: new start with ε to each machine. Wrong guesses just die.
- Final block conditions: start the block only at the beginning or after the other symbol.
- {a^n b^n | n ≤ 3}: 7 states, minimal by a fooling set. b*a* needs 2 states.

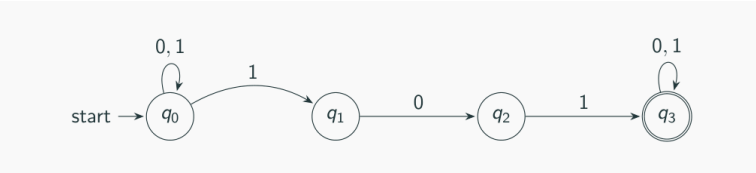
L8 · ε-removal

- E(q) includes q. δ′(q,a) = E(∪δ(E(q),a)). F′ = {q : E(q) ∩ F ≠ ∅}.
- Close before and after reading the symbol.

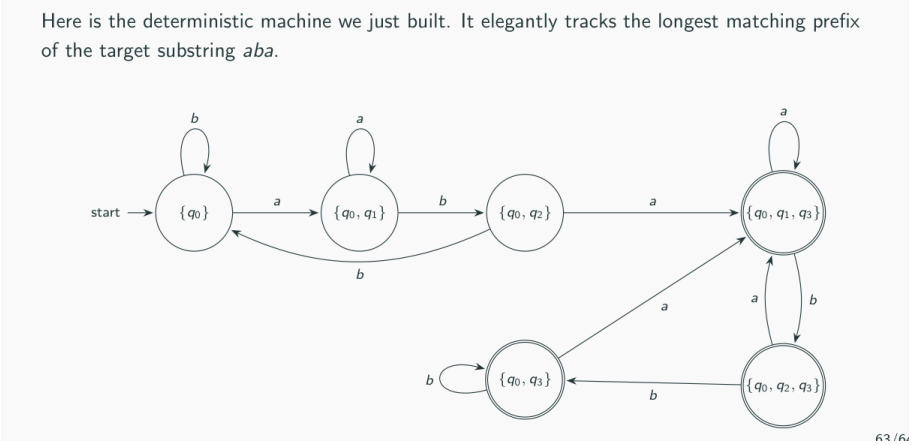
L9 · Subset construction

- Start E({q0}); δ′(S,a) = E(∪δ(S,a)); accept sets meeting F; build reachable only.
- Proof: $\hat{\delta}_D(\{q0\},w) = \hat{\delta}_N(q0,w)$ by induction on |w|.
- n NFA states → between 1 and 2^n DFA states; n-th from end needs 2^n.
- Nondeterminism is inessential (same class), but can be exponentially more succinct.

Key Diagrams to Reproduce



NFA for 'contains 101' (sl. 29) — Fig. 6.1



Subset-construction DFA for 'contains aba' (sl. 108) — Fig. 9.2